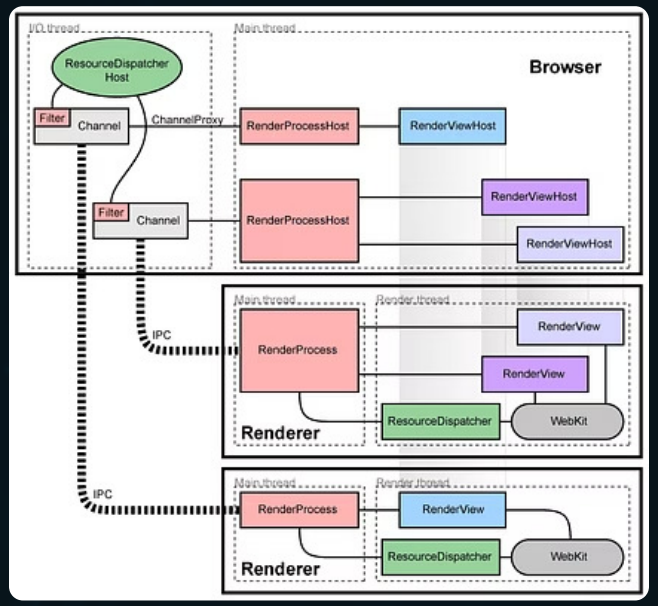
ns-exec

Mediador de ejecución con confinamiento dinámico para Ubuntu (Kubuntu)

PROYECTO FINAL — CRISTÓBAL JOOST



El Problema

23.10 / 24.04 LTS

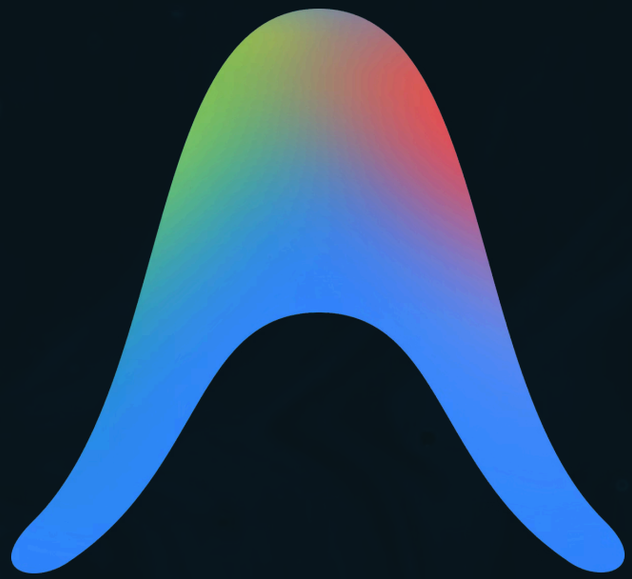
Canonical restringió namespaces no privilegiados vía `apparmor_restrict_unprivileged_userns=1`

Apps afectadas

~~Firefox~~, Chromium, Antigravity y Steam, instalados fuera de rutas estándar.

Resultado

Kernel deniega `clone(CLONE_NEWUSER)` → fallo fatal e cierre inmediato



¿Por qué es relevante?

Soluciones temporales = riesgo

`--no-sandbox`

Desactiva el aislamiento interno de la app

Desactivar variable de AppArmor

Expone todo el sistema a LPE

¿Qué se hizo?

Programa mediador en Python

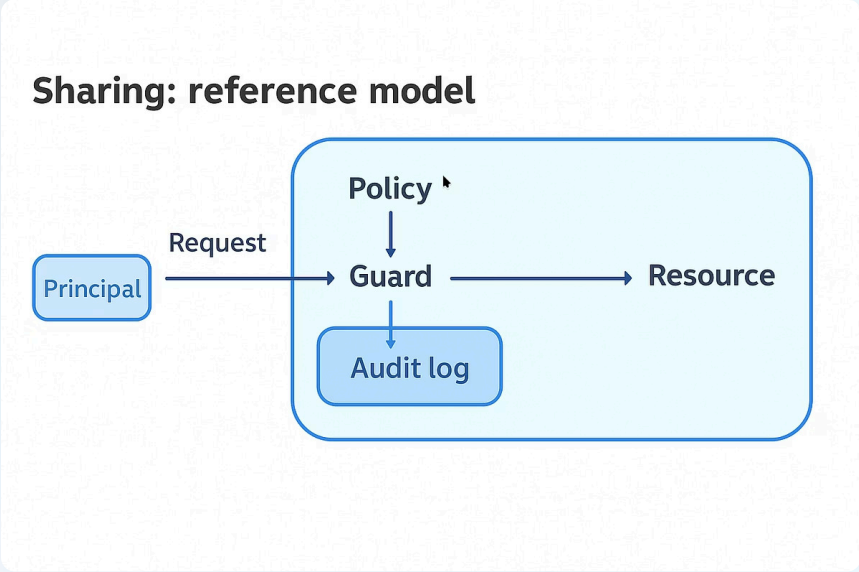
Arquitectura **Guard**: autenticación → autorización → mediación

Lista blanca criptográfica

Hash **SHA-256** de binarios, almacenado en `/etc/users-guard/whitelist.json`

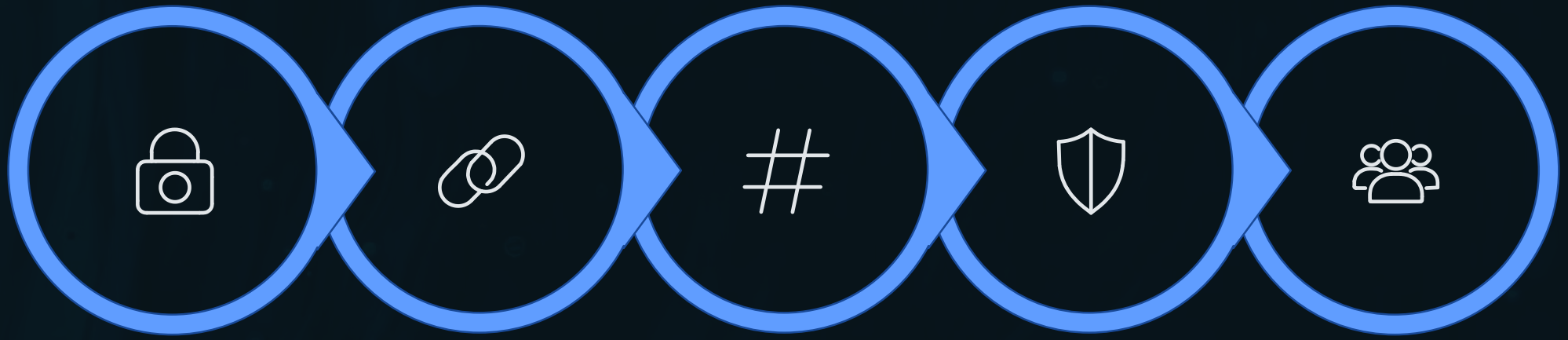
Perfiles AppArmor dinámicos

Inyección directa al kernel vía `apparmor-parser -r` con regla `userns`



```python
#!/usr/bin/env python3
import os
import sys
import hashlib
import json
import subprocess
import pwd

CONFIG_PATH = "/etc/userns-guard/whitelist.json"

def hashPath(bin_path):
    """Calculates the SHA-256 hash of a file by reading it in secure blocks"""
    hash_sha256 = hashlib.sha256()
    try:
        with open(bin_path, "rb") as f:
            for block in iter(lambda: f.read(4096), b""):
                hash_sha256.update(block)
        return hash_sha256.hexdigest()
    except FileNotFoundError:
        print(f"[-] Error: Binary not found at {bin_path}")
        sys.exit(1)

def loadWhiteList():
    """Safely loads the protected JSON whitelist file"""
    try:
        with open(CONFIG_PATH, "r") as f:
            return json.load(f)
    except Exception as e:
        print(f"[-] Failed to read whitelist: {e}")
        sys.exit(1)

def saveWhiteList(data):
    """Saves the whitelist updates"""
    try:
        with open(CONFIG_PATH, "w") as f:
            json.dump(data, f, indent=2)
    except PermissionError:
        print("[-] Error: Permission Denied")
        sys.exit(1)

def loadProfileArmor(profile, path):
    """
    Dynamically generates the AppArmor profile and injects it into the kernel.
    Uses a pipe to apparmor_parser's stdin
    """

    aa_profile = f"""

    #include <tunables/global>

    profile guard_{profile} {path} flags=(attach_disconnected) {{
        include <abstractions/base>

        userns,
        file,
        capability,
        network,
        dbus,
    }}
    """

    try:
        process = subprocess.Popen(
            ["apparmor_parser", "-r"],
            stdin=subprocess.PIPE,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True
        )
        stdout, stderr = process.communicate(input=aa_profile)
        if process.returncode != 0:
            print(f"[-] Error loading AppArmor profile into Kernel: {stderr}")
            sys.exit(1)
        print(f"[+] AppArmor profile 'guard_{profile}' successfully injected")
    except Exception as e:
        print(f"[-] Failure in the AppArmor subsystem: {e}")
        sys.exit(1)

def safeExecution(bin_path, args):
    """
    Drops root permissions by reverting to the original UID/GID and corrects
    shared environment variables
    """
    sudo_uid = os.environ.get("SUDO_UID")
    sudo_gid = os.environ.get("SUDO_GID")

    if not sudo_uid or not sudo_gid:
        print("[-] Error: Original SUDO context not detected.")
        sys.exit(1)

    # 1. Clone current environment for selective cleaning
    clean_env = os.environ.copy()

    # 2. Safely query /etc/passwd to get real user data
    try:
        user_info = pwd.getpwuid(int(sudo_uid))
        real_home = user_info.pw_dir
        real_name = user_info.pw_name
    except KeyError:
        print(f"[-] Error: UID {sudo_uid} does not correspond to a valid system user.")
        sys.exit(1)

    # 3. Correct variables contaminated by the sudo environment
    clean_env["HOME"] = real_home
    clean_env["USER"] = real_name
    clean_env["LOGNAME"] = real_name

    # 4. Strict GID and UID modification of the current process (Dropping privileges)
    os.setgid(int(sudo_gid))
    os.setuid(int(sudo_uid))

    print(f"[+] Privileges degraded to UID {sudo_uid}.")
    print(f"[+] Environment safely sanitized (HOME={real_home}, USER={real_name}).")
    print(f"[+] Executing target binary...")

    # 5. Final invocation passing the corrected environment
    os.execve(bin_path, [bin_path] + args, clean_env)

if __name__ == "__main__":
    # Validate base authentication
    if os.geteuid() != 0:
        print("[-] Authentication Error: This program requires Administrator privileges.")
        print("Usage: sudo yourProgram <path_to_binary> [arguments]")
        sys.exit(1)

    if len(sys.argv) < 2:
        print("[-] Missing parameter. Usage: sudo yourProgram <path_to_binary>")
        sys.exit(1)

    target_bin = sys.argv[1]
    arg_list = sys.argv[2:]

    # Resolve real path (Bypass redirections and symlinks)
    path = os.path.realpath(target_bin)
    app_name = os.path.basename(path)

    print(f"[*] Analyzing binary: {path}")
    calculated_hash = hashPath(path)

    white_list = loadWhiteList()

    # Add bins and authorization
    if path not in white_list:
        # Scenario 1: New Binary
        print(f"[!] WARNING: The binary '{path}' is not registered.")
        confirmation = input(f"Do you want to permanently authorize this application? (SHA256:
{calculated_hash}) [y/N]: ")
        if confirmation.lower() == 'y':
            white_list[path] = calculated_hash
            saveWhiteList(white_list)
            print("[+] Binary added to the whitelist.")
        else:
            print("[-] Execution aborted by the user.")
            sys.exit(1)

    elif white_list[path] != calculated_hash:
        # Scenario 2: Tampering attack or legitimate Update
        print(f"[-] SECURITY ALERT: The binary hash has changed!")
        print("[!] If this is due to an Ubuntu software update, confirm the change.")

        # Since we already guaranteed it is running under sudo, interaction is deliberate
        confirmation = input("Do you want to update the signature in the whitelist and proceed? [y/N]: ")
        if confirmation.lower() == 'y':
            white_list[path] = calculated_hash
            saveWhiteList(white_list)
            print("[+] Signature successfully updated.")
        else:
            print("[-] Security lockout: The modified binary is not authorized to execute.")
            sys.exit(1)
    else:
        print("[+] Cryptographic integrity validation correct.")

    loadProfileArmor(app_name, path)
    safeExecution(path, arg_list)
```

Arquitectura y Diseño



Autenticación

Resolución

Hash SHA-256

Perfil AppArmor

Degradación Ejecución

Flujo de uso práctico

[Ver demostración en YouTube]

01

Configurar infraestructura

Directorio restringido + permisos `chmod 600`

02

Registrar binario

Primera ejecución → confirmación interactiva → firma en whitelist

03

Ejecución transparente

Validación instantánea, política inyectada y app funcionando normal

Resultados

✅ Objetivo cumplido

Apps legítimas ejecutadas sin desactivar seguridad global

🔗 Repositorio público

Código `ns-exec.py` + README + video demostrativo

🔒 Entorno seguro

Desarrollo y pruebas en partición aislada local

Limitaciones identificadas

Confianza en PAM

Sin verificación de **suplantación física** previa

Supply chain

Depende de **librerías** nativas de Python

TOFU estático

Binario malicioso registrado inicialmente será considerado **legítimo**

Defensa en profundidad

Confinamiento granular depende del sandbox interno de la app

Conclusiones y trabajo futuro

Lo aprendido

- Tensión usabilidad ↔️ seguridad en kernel
- Viabilidad de arbitrar acceso dinámicamente
- Defensa en profundidad con MAC + criptografía

Próximos pasos

- Parseo dinámico de políticas de desarrolladores
- Heurística de validación de software
- Mínimo privilegio real: kernel + proceso hijo

Uso de IA

- Gemini
- Gamma

Bibliografía

docs.kernel.org

AppArmor — The Linux Kernel documentation

AppArmor is MAC style security extension for the Linux kernel. It implements a task centered policy, with task "profiles" being created and loaded from user space. Tasks on the system that do not have a profile defined for them run in an...

docs.kernel.org

Linux Security Module Usage — The Linux Kernel documentation

The Linux Security Module (LSM) framework provides a mechanism for various security checks to be hooked by new kernel extensions. The name "module" is a bit of a misnomer since these extensions are not actually loadable kernel modules....

attack.mitre.org

Privilege Escalation, Tactic TA0004 – Enterprise | MITRE ATT&CK®

Privilege Escalation consists of techniques that adversaries use to gain higher-level permissions on a system or network. Adversaries can often enter and explore a network with unprivileged access but require elevated permissions to follow...

https://chromium.googlesource.com/chromium/src/+/0e94f26e8/docs/linuxs andboxing.md

https://man7.org/linux/man-pages/man7/usern amespaces.7.html

https://ubuntu.com/blog/ubuntu-23-10-restricted-unprivileged-user-namespaces